

Advanced Retrieval Techniques

1. Introduction: Why Advanced Retrieval Matters

Simple vector similarity search (dense retrieval) is a good starting point but **rarely sufficient** for production RAG systems.

Core Problem:

Embedding models capture semantic meaning but often fail at:

- Exact keyword matching
- Handling specific constraints (date, category, source)
- Dealing with ambiguous or poorly worded queries
- Ranking the most relevant results at the top

Goal of this session: Learn techniques that significantly improve retrieval precision and recall.

2. Limitations of Pure Vector Search

Key Weaknesses:

- **Keyword Blindness:** Struggles with rare terms, codes, names, or IDs
- **Temporal Issues:** Cannot easily filter recent vs old documents
- **Lack of Structure:** No native support for metadata filtering
- **Ranking Quality:** Top-k results may contain noise
- **Recall vs Precision Tradeoff:** Often sacrifices one for the other

Real-world Impact: Many RAG failures are not due to the LLM but due to poor retrieval.

3. Hybrid Search (Vector + BM25)

Concept:

Combine **Vector Search** (semantic) + **BM25** (keyword-based) search.

How it Works:

- Run both searches in parallel
- Merge and re-score results (Reciprocal Rank Fusion or weighted scoring)

Advantages:

- Vector → Understands meaning and synonyms
- BM25 → Excellent at exact matches and rare keywords

- Overall better recall and precision

Implementation Tip:

Most vector databases (Pinecone, Weaviate, Qdrant, Chroma) support hybrid search natively.

4. Metadata Filtering

One of the most practical and high-ROI techniques.

What is Metadata Filtering?

Applying conditions on structured metadata before or after vector search.

Common Metadata Fields:

- `date` , `year`
- `category` , `department`
- `source` , `document_type`
- `entities` , `author`
- `confidence_score`

Two Strategies:

- **Pre-filtering:** Filter first → then vector search (faster)
- **Post-filtering:** Vector search → then filter

Example Query:

"Show documents about AI regulations from 2025 only, from government sources."

5. Query Expansion Techniques

Why needed?

Users often write short, ambiguous, or suboptimal queries.

Popular Methods:

- **Synonym Expansion:** Add similar words
- **LLM Query Rewriting:** "Rewrite this query for better retrieval"
- **Multi-Query Generation:** Generate 3–5 different versions of the query
- **Hypothetical Document Embeddings (HyDE):** Generate a hypothetical answer and use it for retrieval

Benefit: Dramatically improves recall for complex questions.

6. Reranking with Cross-Encoders

Concept:

Retrieve many documents (e.g., top 50) using fast methods → Re-rank the top results using a more accurate (but slower) model.

Popular Rerankers:

- **Cohere Rerank** (very popular)
- **BGE Reranker**
- **FlashRank** (lightweight & fast)
- **Voyage Rerank, Jina Reranker**

Why it works so well:

Cross-encoders compare query and document **together**, giving much better relevance scoring than bi-encoders.

Typical Flow: Retrieve 20-50 → Rerank top 5-10 → Send to LLM.

7. Late-Interaction Retrievers

Advanced Category (State-of-the-art in many cases).

Main Models:

- **ColBERT** (ColBERTv2): Late interaction for text
- **ColPali**: Late interaction for multimodal (treats PDF pages as images)

Key Idea:

Instead of compressing entire document into one vector, it keeps fine-grained token-level interactions between query and document.

Advantages:

- Much better at matching specific terms
- Excellent for long documents and PDFs
- Higher precision than traditional dense retrieval

Tradeoff: Higher computational cost and slower indexing/retrieval.

8. When to Use Which Technique

Technique	Speed	Accuracy	Cost	Best Use Case
Pure Vector	Fast	Medium	Low	Simple applications

Technique	Speed	Accuracy	Cost	Best Use Case
Hybrid (Vector+BM25)	Fast	High	Low	Most production systems
Metadata Filtering	Very Fast	Very High	Low	When structured data is available
Reranking	Medium	Very High	Medium	When top result quality is critical
Late-Interaction (ColBERT/ColPali)	Slow	Excellent	High	Complex documents, PDFs, high precision

9. Typical Production Retrieval Pipeline

1. **Query Preprocessing** → Query Expansion
2. **Hybrid Search** (Vector + BM25)
3. **Metadata Pre-filtering**
4. **Initial Retrieval** (top 30-50)
5. **Reranking** (Cross-Encoder)
6. **Final Top-k** (usually 5-10) → Send to LLM

This layered approach is used by most advanced RAG systems in 2025.

10. Best Practices & Production Tips

- Always implement **Hybrid Search** as baseline
- Use **Metadata Filtering** aggressively
- Add **Reranking** for final results
- Start simple, then add complexity
- Continuously evaluate retrieval quality (Hit Rate, NDCG, etc.)
- Monitor latency vs quality tradeoff
- Keep the retrieval pipeline configurable
- Test with real user queries regularly

Key Takeaways

- Pure vector search is **not enough** for good RAG.
- **Hybrid Search + Metadata Filtering** gives the biggest immediate improvement.
- **Reranking** is one of the highest ROI techniques.
- **Late-Interaction models** (ColBERT, ColPali) are powerful for complex use cases.
- Great retrieval = Foundation of a great RAG system.

Golden Rule:

Spend more time optimizing retrieval than you think. It often matters more than choosing a bigger LLM.